Sushant Lokhande

San Jose, CA | +16693406655 | lokhandesushant014@gmail.com | sushantlokhande.me | linkedin.com/in/sushantlokhande14 | github.com/sushantlokhande14

Professional Summary

Software Engineer building **reliable backend services, APIs, distributed systems, and data platforms** with Java, Go, Python, SQL, PostgreSQL, and Redis. Experienced in **transaction correctness, idempotency, performance, testing, code review, CI/CD, monitoring, and safe production releases** for high-volume systems.

Technical Skills

Programming: Java, Go, Python, Ruby, JavaScript, TypeScript, SQL
Backend: REST APIs, gRPC, Microservices, Distributed Systems, Async Processing, Background Jobs
Data: PostgreSQL, MySQL, Redis, Kafka, DynamoDB, Transactions, Data Modeling
Reliability: Idempotency, Retry Logic, Rate Limiting, Monitoring, Observability, Incident Debugging
Engineering: Algorithms, Object-Oriented Design, System Design, Unit Testing, Integration Testing, Code Review
Cloud & Delivery: AWS, Docker, Kubernetes, GitHub Actions, CI/CD, Git, Linux

Experience

Trivia Softwares | Software Engineering Intern — Nov 2021 to Jan 2022

- Reduced duplicate transaction writes by **92%** by implementing idempotency keys, database uniqueness constraints, request validation, and retry-safe REST API behavior across payment-style workflows.
- Reduced p95 API latency by **44%** by optimizing PostgreSQL queries, adding indexes and connection pooling, and caching frequently accessed records in Redis.
- Reduced pre-release defects by **39%** by building unit, integration, and API contract tests covering database failures, retries, malformed requests, and concurrent updates.
- Cut deployment validation time by **41%** by automating Docker builds, test execution, configuration checks, and rollback validation through Git-based CI pipelines.

Turito | Software Engineering Intern, Real-Time Systems — Jun 2022 to Oct 2022

- Increased transaction-processing throughput by **2.6x** by partitioning Kafka consumers, introducing asynchronous workers, and batching database writes across services handling more than **3M events per day**.
- Reduced inconsistent application state by **85%** by introducing transactional updates, durable event publishing, retry controls, and reconciliation jobs across multi-service workflows.
- Reduced recurring production incidents by **33%** by instrumenting latency, error rates, queue depth, and dependency health with structured logs, dashboards, and automated alerts.
- Cut release lead time by **38%** by using smaller code changes, peer code reviews, automated tests, staged deployments, health checks, and rollback procedures before production releases.

SJSU | Graduate Assistant — Aug 2025 to May 2026

- Increased distributed job throughput by **2.3x** by implementing parallel workers, partition-aware scheduling, asynchronous I/O, and batched persistence across a multi-node Python service.
- Reduced duplicate and partially completed jobs by **96%** by designing durable task states, idempotent execution, bounded retries, timeout handling, and database-backed deduplication.
- Reduced regression escapes by **42%** by building unit, integration, load, and failure-injection tests for API, database, concurrency, retry, and recovery paths.
- Cut investigation time for distributed failures by **43%** by adding correlation IDs, structured logs, request traces, latency metrics, and replayable failure scenarios across service boundaries.

Relevant Projects

Idempotent Payments API | *Java, Go, PostgreSQL, Redis, Kafka, AWS*

- Prevented duplicate transaction commits in **100% of injected retry tests** by implementing idempotency keys, atomic PostgreSQL transactions, unique constraints, request replay, and durable event publishing.
- Reduced payment-request latency by **49%** by adding Redis caching, connection pooling, batched downstream requests, and indexed PostgreSQL access paths under concurrent load.

Double-Entry Ledger & Reconciliation Service | *Go, PostgreSQL, Kafka, Docker*

- Maintained **100% balanced ledger entries across 10M simulated transactions** by enforcing double-entry invariants, immutable journal records, atomic writes, and deterministic transaction identifiers.
- Reduced reconciliation runtime by **71%** by processing ledger events incrementally, checkpointing completed ranges, batching database reads, and isolating mismatched transactions for targeted replay.

Education

San Jose State University | M.S. in Computer Science — May 2026
University of Mumbai | B.E. in Computer Engineering — May 2023